

FortiFlow

Analyse de logs trafic FortiGate / FortiAnalyzer -- segmentation réseau. Import, matrice de flux, rapprochement configuration/routage, génération CLI FortiGate.

Architecture

Un seul conteneur Docker fortiflow (Node.js + Express, port interne 3737). Image hébergée sur GitHub Container Registry, déployée via Portainer en mode Repository. 13 variables d'environnement, support TLS natif, healthcheck intégré.

FQDN utilisé dans les exemples

Cette procédure utilise fortiflow.monentreprise.lan. Le remplacer partout si le FQDN définitif est différent. Le certificat doit contenir ce nom dans un SAN DNS explicite.

Sommaire

1. Présentation du projet
2. ÉTAPE 1 -- Connexion à la VM de production
3. ÉTAPE 2 -- Création des répertoires persistants
4. ÉTAPE 3 -- Vérification du port 13737
5. ÉTAPE 4 -- Déploiement du stack dans Portainer
6. ÉTAPE 5 -- Vérification du bon fonctionnement
7. ÉTAPE 6 -- Mise en place du HTTPS
8. Mise à jour future
9. Dépannage
10. Checklist finale

Machine	Où ?	Qui ?	Accès
SOURCE (dev)	VPS IONOS	Da Vinci (Hermes)	Automatique (CI/CD)
DESTINATION (prod)	VM Entreprise	Valentin	Console/SSH (root)

Version du document : 3 août 2026

CI : VERTE (4/4) | Image : ghcr.io/tetrax/fortiflow:latest

1. Présentation du projet

FortiFlow est un outil d'analyse de logs trafic FortiGate / FortiAnalyzer pour les prestations de segmentation réseau. Il importe les logs trafic, conserve leur contexte FortiGate/VDOM, construit les matrices de flux, rapproche les observations de la configuration et du routage, puis prépare une CLI FortiGate destinée à être revue par un ingénieur.

- Un seul conteneur Docker : fortiflow (Node.js + Express, port interne 3737).
- Image hébergée sur ghcr.io/tetrax/fortiflow:latest -- déjà buildée par la CI, rien à builder.
- 13 variables d'environnement configurables (PORT, DOMAIN, MAX_UPLOAD_SIZE_MB, etc.).
- Support TLS natif (sans Nginx) ou via reverse proxy Nginx.
- Healthcheck intégré, rate limiting, graceful shutdown.
- CI : build Docker + scan Trivy + push GHCR (4 jobs, tous verts).

Déploiement interne uniquement

L'application ne sera PAS exposée sur Internet. Certificat TLS : PKI entreprise. Nom de domaine : fortiflow.monentreprise.lan.

2. ÉTAPE 1 -- Connexion à la VM de production

Ce que tu vas faire

Te connecter en root sur la VM de l'entreprise où tourne Portainer, et vérifier que Docker et Portainer sont opérationnels.

Commande

```
# Depuis ton poste :  
ssh root@ADRESSE_IP_DE_LA_VM  
  
# Vérifier Portainer :  
docker ps | grep portainer  
# → Doit afficher une ligne  
  
# Vérifier Docker :  
docker version  
# → Doit être ≥ 24.0
```

3. ÉTAPE 2 -- Création des répertoires persistants

Ce que tu vas faire

Créer les répertoires sur l'hôte qui seront montés dans le conteneur. Les conteneurs sont éphémères, les données doivent survivre aux mises à jour. Deux répertoires : /srv/fortiflow/data et /srv/fortiflow/certificates.

Commande

```
mkdir -p /srv/fortiflow/data /srv/fortiflow/certificates
chown -R 1000:1000 /srv/fortiflow
ls -la /srv/fortiflow/
# Doit afficher data/ certificates/ avec 1000:1000
```

UID Docker

Si Docker utilise un UID différent : id docker pour trouver le bon et adapter le chown.

4. ÉTAPE 3 -- Vérification du port 13737

Ce que tu vas faire

Vérifier que le port 13737 (ou 443 si HTTPS direct) est libre sur l'hôte. L'application utilise le port 13737 en externe, mappé vers 3737 dans le conteneur.

Commande

```
ss -tlnp | grep 13737  
# RIEN affiché → OK. Sinon → choisir un autre port.
```

5. ÉTAPE 4 -- Déploiement du stack dans Portainer

Ce que tu vas faire

Créer un stack Portainer en mode Repository. Portainer télécharge automatiquement l'image depuis ghcr.io/tetrax/fortiflow:latest et démarre le conteneur.

Ouvrir Portainer

```
https://ADRESSE_IP_DE_LA_VM:9443
# Se connecter
```

Créer le stack

1

Accéder aux stacks

Menu gauche → Stacks → + Add stack.

2

Nommer le stack

Name : fortiflow

3

Configurer le Repository

Build method : Repository

4

URL et référence

Repository URL : <https://github.com/Tetrax/FortiFlow>

Repository reference : refs/heads/main

Compose path : docker-compose.portainer.yml

5

Renseigner les variables d'environnement

Section Environment variables → + Add environment variable pour chaque ligne du tableau ci-dessous.

Nom	Valeur
PORT	3737
DOMAIN	devval.com
MAX_UPLOAD_SIZE_MB	2048
MAX_DECOMPRESSED_SIZE_MB	4096
MAX_ARCHIVE_ENTRIES	100
MAX_XLSX_SIZE_MB	100
MAX_WORKSPACE_UNCOMPRESSED_MB	1024
MAX_SESSION_DEDUPE_KEYS	2000000
MAX_ANALYSIS_WORKERS	1
MAX_ANALYSIS_QUEUE	3
ANALYSIS_WORKER_MEMORY_MB	0
SSL_KEY	/certs/privkey.pem
SSL_CERT	/certs/fullchain.pem

Ne pas dupliquer `SSL_KEY` / `SSL_CERT`

Ces variables sont déjà dans le compose. Les renseigner dans Portainer uniquement si tu les surcharges avec des valeurs différentes.

6

Déployer

Cliquer Deploy the stack. Portainer : 1) Pull ghcr.io 2) Crée le conteneur 3) Démarre (~15s).

Fichier compose

Le fichier `docker-compose.portainer.yml` référence l'image `ghcr.io/tetrax/fortiflow:latest`. Portainer la télécharge automatiquement. Aucun build local nécessaire.

6. ÉTAPE 5 -- Vérification du bon fonctionnement

Ce que tu vas faire

Vérifier que le conteneur est bien démarré, que l'application répond en HTTP, et que le dashboard est accessible depuis le réseau interne.

Depuis Portainer

1 Contrôler le conteneur

Dans Containers, le conteneur fortiflow doit afficher running (healthy).

Depuis le terminal SSH root

2 Vérifier avec Docker

Trois contrôles rapides pour valider le déploiement.

```
docker ps --filter "name=fortiflow"
# → Doit afficher le conteneur avec statut healthy

curl -s http://localhost:13737/ | head -10
# → Doit afficher du HTML avec "FortiFlow"

docker logs fortiflow --tail 30
# → Vérifier l'absence d'erreurs critiques
```

Depuis un navigateur

3 Test navigateur

Depuis un poste du réseau interne, ouvrir `http://ADRESSE_IP_DE_LA_VM:13737`. Le dashboard FortiFlow doit s'afficher.

7. ÉTAPE 6 -- Mise en place du HTTPS

Ce que tu vas faire

Configurer le certificat TLS interne pour que l'application soit accessible en HTTPS sur `https://fortiflow.monentreprise.lan`. Deux options : avec Nginx (recommandé en entreprise) ou TLS direct dans l'application (sans Nginx).

PKI interne obligatoire pour .lan

Let's Encrypt ne signe normalement pas un suffixe privé .lan. La CA interne doit être distribuée aux postes, typiquement par GPO ou MDM.

OPTION A -- Avec Nginx (recommandé en entreprise)

Obtenir le certificat

Demander à l'équipe IT les deux fichiers pour `fortiflow.monentreprise.lan` (ou wildcard `*.monentreprise.lan`) :

- Certificat public : `fortiflow.lan.crt`
- Clé privée : `fortiflow.lan.key`

Transférer vers la VM

```
scp fortiflow.lan.crt root@VM_IP:/srv/fortiflow/certificates/  
scp fortiflow.lan.key root@VM_IP:/srv/fortiflow/certificates/
```

Placer les certificats

```
cd /srv/fortiflow/certificates  
ls -la # Doit afficher fortiflow.lan.crt et fortiflow.lan.key  
chmod 600 fortiflow.lan.key  
chmod 644 fortiflow.lan.crt  
chown -R 1000:1000 /srv/fortiflow/certificates
```

Configurer Nginx

Créer le fichier de configuration du site :

```
nano /etc/nginx/sites-available/fortiflow
```

Coller ce contenu :

```
server {
    listen 443 ssl http2;
    server_name fortiflow.monentreprise.lan;

    ssl_certificate      /srv/fortiflow/certificates/fortiflow.lan.crt;
    ssl_certificate_key  /srv/fortiflow/certificates/fortiflow.lan.key;

    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers HIGH:!aNULL:!MD5;

    client_max_body_size 2048m;
    proxy_read_timeout 3600s;

    location / {
        proxy_pass http://127.0.0.1:13737;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto https;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
    }
}
```

Activer le site

```
ln -s /etc/nginx/sites-available/fortiflow /etc/nginx/sites-enabled/
nginx -t && systemctl reload nginx
```

DNS interne

Demander à l'équipe IT un enregistrement DNS interne :

```
fortiflow.monentreprise.lan → IP_VM
```

En attendant le DNS

Ajouter IP_VM fortiflow.monentreprise.lan dans /etc/hosts de ton poste pour tester.

OPTION B -- TLS direct sans Nginx (alternative)

Principe

L'application gère elle-même le TLS. Les certificats sont placés dans /etc/ssl/fortiflow/ sur l'hôte, montés en /certs/ dans le conteneur (lecture seule). Le serveur détecte automatiquement leur présence et bascule en HTTPS.

Placer les certificats

```
mkdir -p /etc/ssl/fortiflow/

# Copier la clé privée et le certificat
cp /chemin/vers/privkey.pem /etc/ssl/fortiflow/privkey.pem
cp /chemin/vers/fullchain.pem /etc/ssl/fortiflow/fullchain.pem

chmod 600 /etc/ssl/fortiflow/privkey.pem
chmod 644 /etc/ssl/fortiflow/fullchain.pem
chown -R 1000:1000 /etc/ssl/fortiflow/
ls -la /etc/ssl/fortiflow/
# Doit afficher privkey.pem et fullchain.pem avec 1000:1000
```

Variables d'environnement

Vérifier que les variables TLS sont bien présentes dans le stack :

```
SSL_KEY=/certs/privkey.pem
SSL_CERT=/certs/fullchain.pem
```

Redéployer

1

Mettre à jour le stack

Dans Portainer : Stacks → fortiflow → Pull and redeploy → Update.

Important

Avec TLS direct, l'application écoute en HTTPS sur le port 13737 (plus HTTP). Le test curl doit devenir : curl -k https://localhost:13737

8. Mise à jour future

Ce que tu vas faire

Mettre à jour l'application vers la dernière version disponible sur GHCR. Deux méthodes : Portainer (recommandé) ou SSH (plan B).

Portainer (recommandé)

1

Pull and redeploy

Dans Portainer : Stacks → fortiflow → Pull and redeploy → Update. Portainer pull la dernière image ghcr.io/tetrax/fortiflow:latest et redémarre (~5 sec d'interruption).

SSH (plan B)

```
cd /srv/fortiflow
docker compose -f docker-compose.portainer.yml pull
docker compose -f docker-compose.portainer.yml up -d
```

Le compose référence ghcr.io directement → docker compose pull fonctionne.

9. Dépannage

Problème	Vérification / Action
Stack ne se déploie pas	Vérifier toutes les variables : PORT, DOMAIN, MAX_UPLOAD_SIZE_MB, etc.
Conteneur en erreur	chown -R 1000:1000 /srv/fortiflow puis docker logs fortiflow
Statut healthy absent	Vérifier que le healthcheck est présent dans le compose
HTTPS ne fonctionne pas	Vérifier les chemins dans Nginx, nginx -t, vérifier le DNS interne
Certificat non reconnu	Vérifier que le poste fait confiance à la CA racine de l'entreprise
Dashboard page blanche	docker logs fortiflow --tail 50, vérifier les variables d'environnement
Erreur ANALYSIS_CANCELLED	Analyse interrompue -- relancer l'upload
Erreur DECOMPRESSED_SIZE_LIMIT	Archive trop volumineuse -- augmenter MAX_DECOMPRESSED_SIZE_MB
Portainer : no such image	Utiliser la méthode Repository (pas d'import manuel)
Permission denied sur /certs	chmod 644 sur les .pem dans /etc/ssl/fortiflow/

10. Checklist finale

- /srv/fortiflow/ créé avec permissions/ownership 1000:1000
- Port 13737 libre (ss -tlnp | grep 13737)
- Stack Portainer déployé, conteneur healthy
- curl http://localhost:13737 fonctionne
- Certificats dans /srv/fortiflow/certificates/ (Nginx) ou /etc/ssl/fortiflow/ (TLS direct)
- Nginx configuré pour HTTPS sur fortiflow.monentreprise.lan (Option A)
- DNS interne créé (fortiflow.monentreprise.lan → IP_VM)
- https://fortiflow.monentreprise.lan accessible, cadenas vert

Déploiement terminé

Application accessible en HTTPS sur le réseau interne. Repo : <https://github.com/Tetrax/FortiFlow> | Image : ghcr.io/tetrax/fortiflow:latest | CI : <https://github.com/Tetrax/FortiFlow/actions>